

How the system is put together

Architecture, the data model, how a visit is priced, how package days are spent exactly once, how the security boundary actually works — and the traps that cost real time to find.

Dog daycare management system

Engineering documentation · revision 2

Contents

1 Architecture

One Next.js app, three audiences

2 Routing

Which half of the app a path belongs to

3 The data model

Fourteen tables and what keys them

4 A visit, end to end

From a phone number to a priced record

5 Pricing

One function, one breakdown, no second copy

6 Package accounting

Why a day can only be spent once

7 Boarding walks

The one place a block is spent in bulk

8 Runtime theming

One colour becomes an interface

9 Settings as data

White-labelling without a build

10 Security

Accounts, sessions and row-level security

11 Card payments

Point of Sale deep links, and surviving the hand-off

12 Requests and approval

Queues that fan out into real records

13 Time zones

The bug this project keeps re-learning

14 Print

Screens that have to become paper

15 Testing and fixtures

Including how this document was made

16 Known limitations

Stated rather than discovered

1. Architecture

One Next.js 14 application, App Router, TypeScript throughout. It serves three audiences from one deployment and one database.

Layer	Technology	Notes
Framework	Next.js 14.2 (App Router)	Route groups separate marketing from app
Language	TypeScript 5.5	<code>strict</code> ; no <code>any</code> in the domain layer
Styling	Tailwind 3.4 over CSS custom properties	Colours are <code>rgb(var(--x) / <alpha-value>)</code> , so themes change at runtime
Data	Supabase JS against PostgREST	Called from the browser, not through an API layer
Auth	Supabase Auth, email + password	Sessions persisted; no sign-up route exists
Email	Resend, via one server route	The only server-side code in the project
Payments	Square Point of Sale API	Deep links — see section 11
Hosting	Vercel	Static marketing pages, client-rendered app pages

Why the browser talks to the database directly. There is no bespoke API tier. PostgREST plus row-level security *is* the API, and the policies are the authorisation layer. Adding a Node middle tier would have meant re-implementing those checks in a second place, where they could drift. The cost is that every query is visible to a determined client — which is exactly why section 10 matters.

2. Routing

Three groups, distinguished in `lib/routes.ts` rather than by convention, because several behaviours key off which group a path is in.

Group	Paths	Behaviour
Marketing	<code>app/(site)/</code> — <code>/</code> , <code>/prices</code> , <code>/boarding</code> , <code>/dog-daycare</code> , <code>/dog-walking</code> , <code>/bath</code> , <code>/gallery</code> , <code>/about-us</code> , <code>/contact</code>	Public. Owns its own page titles for SEO. Never dark.
App, public	<code>/kiosk</code> , <code>/enroll</code> , <code>/book</code> , <code>/signup</code>	Runs on somebody else's screen. Gets the runtime business name; never dark.
Back office	<code>/dashboard</code> , <code>/in-house</code> , <code>/calendar</code> , <code>/stay-report</code> , <code>/day-report</code> , <code>/packages</code> , <code>/requests</code> , <code>/settings</code> , <code>/dogs/[id]</code> , <code>/owners/[phone]</code>	Behind <code>StaffGate</code> . The only place dark mode applies.

An allowlist, not a blocklist. Dark mode asks "is this a staff route?" rather than "is this public?". The marketing site will grow pages, and each new one would otherwise have to remember to opt out. The same list stops the runtime white-label title from overwriting the marketing pages' SEO titles — a bug that briefly had every public page reporting itself as the sign-in screen.

Renamed routes keep redirect stubs: `/records` → `/in-house`, `/boardings` → `/calendar`, `/report` → `/stay-report`, `/daily` → `/day-report`. Query strings are carried across, because printed sheets and lobby-tablet bookmarks contain the old paths.

3. The data model

Table	Keyed by	Holds
<code>dogs</code>	<code>id</code> ; <code>phone</code> groups a household	Profile, breed, temperament, health, waiver, pinned default packages, <code>meet_greet_on</code>
<code>owners</code>	<code>phone</code>	Contact, address, emergency contact, vet
<code>signins</code>	<code>id</code> , <code>dog_id</code>	One row per drop-off or pick-up. Action, service, add-ons, price, who, walk fields
<code>boardings</code>	<code>id</code>	A reservation: dates, add-ons, walks per day, instructions, <code>walk_package_id</code>
<code>packages</code>	<code>id</code> , <code>phone</code>	A block. <code>kind</code> is daycare or walk; <code>dog_name</code> null means shared
<code>package_uses</code>	<code>package_id</code> , <code>signin_id</code>	The ledger. One row per use — the audit trail behind a deduction
<code>payments</code>	<code>phone</code>	Amount, method, note, date
<code>vaccinations</code>	<code>dog_id</code>	Vaccine, given, expires
<code>walk_logs</code>	(<code>boarding_id</code> , <code>date</code> , <code>walk_index</code>) unique	Out, back, who — per stay per day per slot
<code>meal_logs</code>	(<code>boarding_id</code> , <code>date</code> , <code>meal_type</code>)	Fed, by whom
<code>enrollments</code>	<code>id</code>	A submitted enrollment; the form itself in <code>data</code> jsonb
<code>boarding_requests</code>	<code>id</code>	A requested stay; the form in <code>data</code> jsonb
<code>site_photos</code>	<code>slot</code>	Hero, gallery and team images for the website
<code>settings</code>	single row, <code>id = 1</code>	Every business-specific value, as one jsonb blob with <code>updated_at</code>

TWO DECISIONS WORTH NAMING

Sign-ins are append-only events, not a visit row with two timestamps. A visit is derived by walking the history: the last drop-off with no pick-up after it is an open visit. That makes a boarding stay spanning days fall out naturally, and it means a correction is a new row rather than a mutation.

A submitted form is stored whole, as jsonb. The queue tables carry the columns staff need to triage — name, dates, status — and the entire submission alongside them. The approval path reads the blob, so adding a question to the form does not need a migration.

4. A visit, end to end

`lib/signin.ts` is the only place a sign-in is written. Both the lobby kiosk and the staff front-desk panel go through it, so pricing and deduction cannot drift between them.

#	Step	Where
1	Phone number typed; one round trip fetches dogs, packages, boardings and 300 rows of history	<code>loadPhoneContext</code>
2	History is walked into open visits — last drop-off per dog with no later pick-up	<code>buildOpenVisits</code>
3	Service is decided per dog: at pick-up locked to how it came in; at drop-off a reservation covering today forces boarding	<code>effectiveService</code>
4	Add-ons chosen, or pre-filled from the reservation	kiosk / front desk
5	At pick-up, whether a package applies is decided — full days only	<code>packageApplies</code>
6	Price is estimated into a labelled breakdown	<code>estimatePrice</code>
7	The <code>signins</code> row is inserted and its id captured	<code>performSignIn</code>
8	Package uses are adopted or consumed, tied to that id	<code>adoptExistingUse</code> / <code>consume</code>
9	A boarding stay's walk block is spent in one go	<code>input.boardingWalks</code>

5. Pricing

`lib/pricing.ts` returns a structured breakdown, never a bare number, so every screen can show the same itemised total and none of them re-derives it.

Rule	Detail
Full vs half day	Four hours or less is a half day. Configurable
Package coverage	Only ever the full-day base rate; add-ons still bill on top
Boarding	Per night, plus a late-pickup fee charged once, on the last day only

Rule	Detail
Boarding add-ons	Walks per walk (per day × nights); medication per night; nail trim once
Bath	Priced by size. A walk-in bath has no size until staff set one, so no charge
Covered lines	Shown as an explicit \$0 rather than dropped, so nothing looks missing
Packages sold today	Part of today's total — but only today's

Prices are read live from settings through getter objects, so `PRICING.daycareFullDay` still works at every call site while the value comes from the database. Every pricing function stays pure and synchronous.

A mid-stay estimate is explicitly told that its pick-up time is a stand-in, because a boarding stay has not earned the last-day late fee just because the clock passed noon on day two.

6. Package accounting

The hard requirement: a visit spends **one** day, and there is an audit row for it.

The subtlety is that staff can pick a block on the walk log *before* the dog is signed out, and that pick deducts immediately so the count visibly moves. When the dog is then signed out, a second deduction must not happen.

Mechanism	What it does
<code>adoptExistingUse</code>	Looks for an unlinked use for this dog, today, of the right kind. If found, it is linked to the new sign-in rather than a fresh one being created
Matched on kind , not id	If staff earmarked block A and the default resolves to block B, the visit still owes one walk. Taking one from each would charge twice, and staff's explicit choice should win
<code>signin_id</code> on the ledger	Pins a use to one exact visit. Rows predating the column resolve to null rather than being guessed at
Reassignment refunds	Moving a visit to a different block refunds the old, deducts the new and repoints the ledger row, so history stays truthful

7. Boarding walks

A daycare visit spends one walk at pick-up. A boarding stay spends *all* of its walks at pick-up, because that is when a stay is billed — `walksCoveredFor` caps the count by what the block has left and by how many walks the stay actually has, so two overlapping stays cannot spend the same uses.

A bug worth recording. A boarding drop-off copies the walk add-on across from the reservation. That made the per-visit walk deduction fire as well as the whole-stay one, and a four-night stay with two walks a day spent seven walks instead of six. The per-visit path is now daycare-only; boarding settles exclusively through the reservation's block.

The display side of the same problem: between pointing a stay at a block and the dog going home, those walks are promised but the ledger has not moved. A picker reading *10 of 10 left* invites staff to promise the same walks twice, so the label subtracts a **held** count — display only, with the ledger still the record of what was spent.

The screenshot shows a web application interface for a 'Walk log'. At the top, there's a navigation bar with buttons like 'Dashboard', 'In House', 'Calendar', 'Packages', 'Requests', 'Day report', and 'Settings'. Below this, the 'Walk log' section is active, showing a date picker for '2026-08-10' and buttons for 'Sign-in list', 'Sign a dog in / out', and 'Print / Save as PDF'. The main content is a table with the following columns: 'DOG', 'WALK', 'OUT -> BACK', 'BY', and 'WALK PACKAGE'. The table is divided into sections for 'DAYCARE' and 'BOARDING'. Under 'DAYCARE', there are two rows: one for 'Nala' (Tom Whitfield) with a walk from 1:00pm to 1:30pm by 'Luis C.', and another for 'Buki' (Ana Alvarez) with a walk from 11:00am to 11:30am by 'Rosa M.' and a 'No walk package' dropdown. Under 'BOARDING', there are two rows for 'Otis' (Brennan). The first row is 'Walk 1 of 2' from 9:00am to 9:30am by 'Rosa M.', with a dropdown showing 'Otis - 4 of 10 walks left (6 held)' and a note '6 walks held for this stay, spent at pick-up'. The second row is 'Walk 2 of 2' from 3:00pm to 3:30pm by 'Luis C.'.

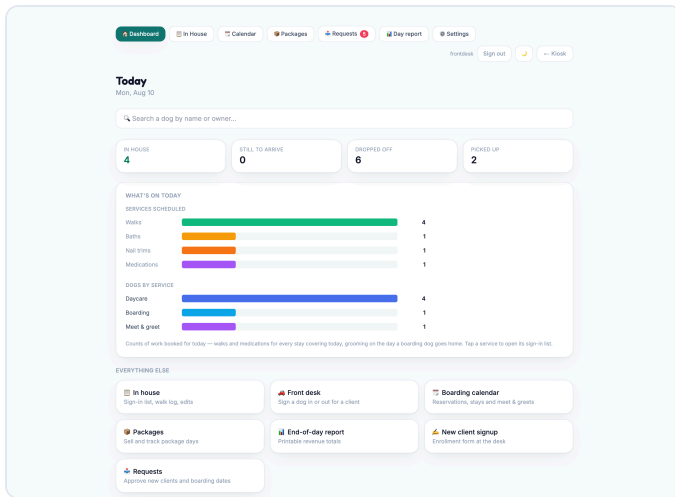
The walk log. **4 of 10 walks left (6 held)** for a four-night stay at two walks a day, with the picker rendered once per stay rather than beside every slot.

8. Runtime theming

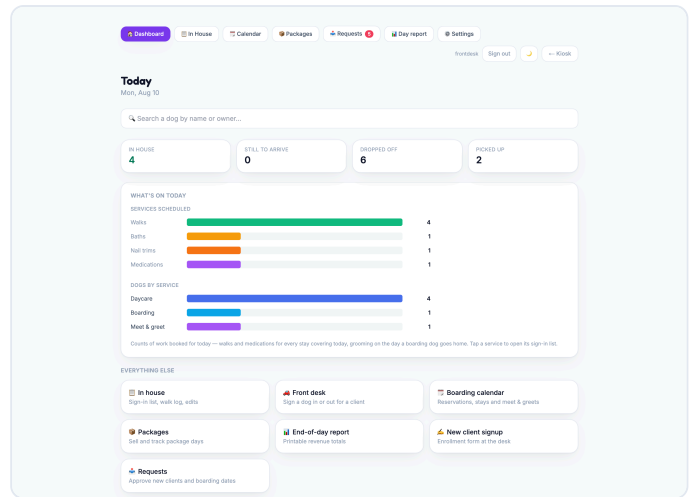
Tailwind colours are declared against CSS custom properties as `rgb(var(--accent-500) / <alpha-value>)`, which keeps opacity modifiers like `bg-accent-500/60` working while the underlying value is set at runtime.

Staff pick **one** brand colour. `lib/theme.ts` derives the ramp by mixing toward white and black — asking a front desk to choose `accent-400` would be absurd, and a hand-picked ramp is where off-brand tints creep in.

Button text is computed, not chosen. `readableInk` takes the WCAG relative luminance of the brand colour and returns whichever of white or near-black has the better contrast ratio, exposed as `--accent-ink`. A pale brand colour therefore gets dark text instead of unreadable white.



#0f766e



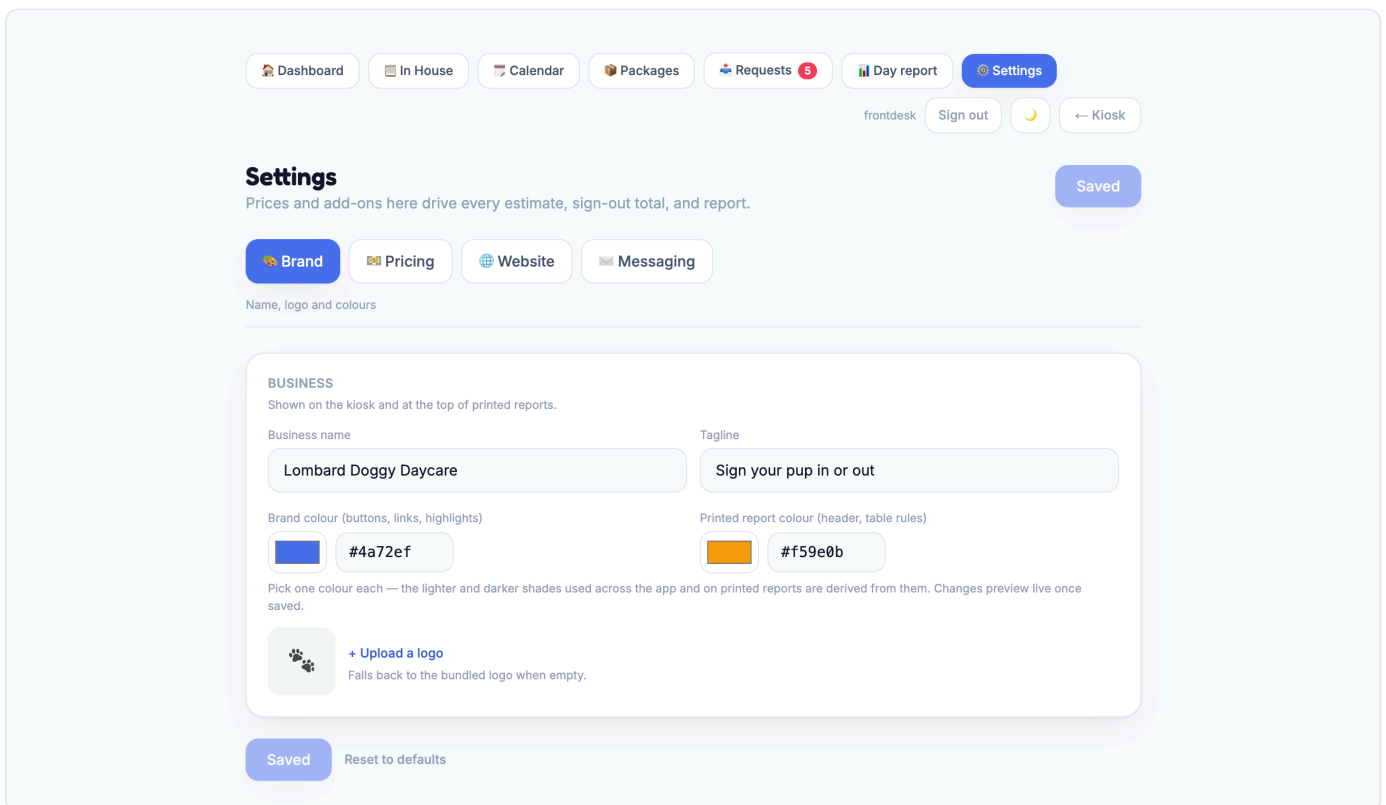
#7c3aed

Same build, same CSS, one value different in the settings row.

9. Settings as data

One row, `id = 1`, holding a jsonb blob. `SettingsProvider` loads it once at startup and hydrates a module-level cache that the pricing getters read.

Children render immediately against shipped defaults rather than blocking on the network — the kiosk has to come up even when the database is slow — and re-render when the real values land. Saving compares `updated_at` against what was loaded and raises a conflict rather than silently overwriting another tab's edit.



Settings → Brand. The staff-name list here is what the walk log's *By* dropdown offers, which is how free-text initials were retired.

10. Security

What was there first: a passcode compared in the browser against a `NEXT_PUBLIC_` environment variable — so it shipped in plain sight inside the JavaScript bundle, and the database was never asked anything.

What is there now:

Layer	Mechanism	What it actually stops
Sign-in	Supabase Auth session	The database refuses the request, not just the UI
Row-level security	<code>rls-lockdown.sql</code>	The anon key can read <code>settings</code> and <code>site_photos</code> and insert into the two queues. Nothing else
Kiosk session	Its own account	The lockdown would otherwise have to leave every table open for the kiosk to work
Idle lock	30 minutes, sliding	An unattended back-office screen. Convenience, not a boundary — it does not end the session
No sign-up route	Accounts created in Supabase by hand	Self-service access

The ordering hazard. Running the lockdown before the accounts exist and the signing-in code is deployed takes the app offline instantly, and it looks like the migration broke the database. Create users, deploy, then lock down.

Outstanding. `.env.local` still carries a service-role key under a `NEXT_PUBLIC_SUPABASE_SECRET_KEY` name. That prefix inlines the value into the client bundle, and a service-role key bypasses row-level security entirely. It should be renamed without the prefix or deleted.

11. Card payments

The daycare uses a Square Reader paired to a phone. Readers cannot be driven remotely — Square's Terminal API only pushes a cart to Terminal or Register hardware — so this uses the **Point of Sale API** instead: the browser hands off to the Square app with the amount pre-filled, the client taps their card there, and Square returns to a callback page with the result.

iOS takes a custom scheme (`square-commerce-v1://`) carrying JSON; Android takes an `intent:` URL with namespaced extras. Same information, entirely different syntax.

Consequence of the hand-off	How it is handled
It only works where the Square app is installed	A desktop browser says so rather than opening a dead link

Consequence of the hand-off	How it is handled
The page is navigated away from entirely	What the payment is for is written to <code>localStorage</code> first, keyed by a token Square echoes back. A kiosk sign-out is submitted <i>before</i> the hand-off
Square takes one amount, not a cart	The itemised breakdown travels in the note and lands on the seller receipt, truncated at 500 characters because a rejected charge at the counter is worse than a shortened receipt line
The callback can fire more than once	The transaction id makes the ledger write idempotent
The stash is consumed on first use	So a reload lands with no context. It checks the ledger before reporting a problem — telling staff a recorded payment was lost would cause a duplicate entry

Test mode skips Square entirely and calls the return page with the result Square would have sent. Everything downstream — matching the stash, the duplicate check, the ledger write — runs for real, which is the part worth testing. Simulated rows are labelled and removable in bulk.

12. Requests and approval

Both queues follow the same shape: insert a row with the whole submission, notify staff, and do nothing else until somebody approves.

	Enrollment	Boarding request
Approval creates	A dog per name, plus owner and vaccination rows	A reservation per dog
Pre-approval check	Vaccination dates are mandatory on the form itself	Profile, waiver and vaccination status resolved live
Ordering	Newest first	Soonest drop-off first — a stay next week beats one in three months
Flags	—	Short notice, and dates already passed
Partial success	—	A dog with no profile still books, and says so

The eligibility panel. The client's own *already enrolled?* answer is treated as a claim; this is what the database actually has.

13. Time zones

Postgres `current_date` is UTC. The daycare's day is **America/Los_Angeles**. This project has re-learned that at least four times: seeded data landing on tomorrow, evening sign-ins filed against the next day, and a stay report listing an arrival on the wrong date because it read `created_at.slice(0, 10)` — which is the UTC day, not the local one.

The rule: `lib/dates.ts` owns day keys, and every comparison against a `date` column goes through it. `slice(0, 10)` on a timestamp is always a bug in this codebase.

14. Print

Two screens exist to become paper: the stay report and the day report. Print styles drop navigation, add a header band with the business name, and turn empty inputs into ruled lines so a half-filled log is still writable by hand.

Width is the recurring problem. The stay report's walk log originally laid one column out per walk, each holding three controls — so two or three walks a day ran off the page. Walks now run *down* the page, which makes the width independent of how many walks a stay books. Wide tables also scroll inside their own box rather than pushing the page sideways on a tablet.

Boarding stay report

Edit reservation
Delete
Print / Save as PDF

← Search a different number



Otis

Brennan · (415) 555-0198

Full profile, vaccines & history →

STAY

Mon, Aug 10 → Thu, Aug 13

3 nights

Walks

Medication

Feeding

1 cup am, 1 cup pm — own food, in the blue tub

Medication

Half a tablet with breakfast, wrapped in cheese

Walks

2 per day

Notes

Sleeps better with the radio on.

WALK LOG — 2x/DAY

Date	Walk	Out → back	By
Mon, Aug 10	1 of 2	9:00am → 9:30am	Rosa M.
	2 of 2	3:00pm → 3:30pm	Luis C.
Tue, Aug 11	1 of 2	9:00am → 9:30am	Rosa M.
	2 of 2	— → —	—
Wed, Aug 12	1 of 2	— → —	—
	2 of 2	— → —	—
Thu, Aug 13	1 of 2	— → —	—
	2 of 2	— → —	—

Saves as you type. Anything left blank prints as a line to fill in by hand.

MEAL LOG

Date	Breakfast	Lunch	Dinner	Snack
Mon, Aug 10	☒ Rosa M.	☐	☒ Luis C.	☐
Tue, Aug 11	☒ Rosa M.	☐	☐	☐
Wed, Aug 12	☐	☐	☐	☐
Thu, Aug 13	☐	☐	☐	☐

Tap a box to mark a meal fed.

SIGN IN / OUT TIMES

The stay report as it prints. Walk log, meal chart and charges all fit the page width regardless of the stay's shape.

15. Testing and fixtures

The pure domain functions — template rendering, age from birthdate, fixed status, night counts, price breakdowns — are exercised directly, because they are where the business rules live and they have no dependencies to stand up.

HOW THIS DOCUMENT WAS PRODUCED

Worth recording, because it is reusable. The manuals are built from an out-of-tree copy of the app with three patches applied:

- `lib/supabase.ts` is replaced by a fixture client — canned data, but with filters genuinely honoured, so an owner profile does not show every dog in the fixture.
- `StaffGate` and `KioskGate` render their children immediately.
- The idle lock never fires mid-capture.

Three reasons it is a fixture rather than the live database: real dogs, owners and phone numbers must not end up in a PDF that gets emailed to a prospect; screens photograph differently depending on the day, so a manual would go stale when a stay ended; and the documentation build has no business holding the back-office password.

Screenshots are then driven over the DevTools Protocol, which is what makes seeding the dark-mode preference, waiting on a specific element and capturing a full page possible at all. Every patch asserts before it writes, so a rename upstream fails loudly rather than producing a manual full of login screens.

16. Known limitations

Limitation	Why it is acceptable today	What it would take
In House lists today's sign-ins , so a dog mid-stay does not appear on the day after drop-off	The screen is the sign-in record for a date; the calendar and stay report cover stays in progress	Merge active boardings into the board as synthetic rows
A boarding pick-up's recorded price and the reservation's own total are computed by different paths	The reservation total is the invoice, and it is what staff quote	Bill a stay once, from the reservation, and record a reference rather than a price
No offline mode	The lobby tablet is on the shop's own wi-fi	A service worker and a write queue, plus conflict rules
No role separation	Every staff account can reach Settings	A role claim on the account plus policies per table
Single location	The business has one	A location column on most tables, and a picker
Client-side queries	Row-level security is the authorisation layer	Nothing, unless business logic has to be hidden rather than just enforced